SnapStart Performance Optimization for Serverless Machine Learning: Structural Decomposition, Statistical Analysis, and SLO-Aware Configuration

A Comprehensive Empirical Study of Machine Learning Inference Workloads

Vishu Porwal
Department of Computer Science
K.R. Mangalam University
Gurugram, India
2201010008@krmu.edu.in

Alok Kumar Jaiswal
Department of Computer Science
K.R. Mangalam University
Gurugram, India
2201010016@krmu.edu.in

***Abstract*—Serverless computing has emerged as a transformative paradigm for cloud-native applications, yet cold start latency remains a critical performance bottleneck limiting adoption for latency-sensitive workloads. AWS Lambda SnapStart addresses this challenge through pre-initialized execution environment snapshots, but comprehensive empirical analysis of its effectiveness remains limited. This paper presents the first rigorous performance and cost-efficiency analysis of Lambda SnapStart for machine learning inference workloads, with systematic statistical validation. We conducted extensive experiments comprising 438 function invocations across three memory configurations (512MB, 1024MB, 2048MB), evaluating cold start elimination, warm start performance, sustained load behavior, and SLO compliance. Results demonstrate that SnapStart achieves complete cold start elimination (100% reduction, 7094ms $\rightarrow$ 0ms) for configurations $\geq$1024MB and 68.72% reduction (7069ms $\rightarrow$ 2211ms) at 512MB, with high statistical significance (t = 5.29, p$<$0.001, Cohen's d = 0.51). Under 10-minute sustained load (198 invocations), SnapStart demonstrates 29.5% improvement in P95 latency (2478ms $\rightarrow$ 1748ms) and 40.9% improvement in P99 latency (4607ms $\rightarrow$ 2723ms) compared to baseline functions. Through structural decomposition analysis, we identify network overhead as the dominant latency component ($>$85% of end-to-end latency), challenging conventional optimization focus on initialization time reduction. Pareto frontier analysis and cost decomposition reveal 4.26% cost increase ($0.43 per 10M invocations) for 33.47% performance improvement, yielding a 7.8$\times$ cost-performance efficiency ratio favorable for latency-sensitive applications. Additionally, we evaluate SLO compliance across five thresholds (100ms–5s), demonstrating SnapStart's particular effectiveness for 2–5 second requirements (71.4% vs. 64.3% compliance at 5s). Statistical validation through independent samples t-tests, Mann-Whitney U tests, and effect size calculations confirms robustness of findings. These results provide actionable, evidence-based guidance for practitioners adopting serverless ML inference and establish a methodological framework for evaluating serverless optimization techniques through the largest empirical Python SnapStart dataset to date.**

***Index Terms*—serverless computing, AWS Lambda, SnapStart, cold start, machine learning inference, performance optimization, cost analysis, SLO compliance**

I. Introduction

Serverless computing has revolutionized cloud application development by abstracting infrastructure management and enabling event-driven, pay-per-use execution models [1]. AWS Lambda, as a leading Function-as-a-Service (FaaS) platform, processes billions of invocations monthly, supporting diverse workloads from web APIs to data processing pipelines [2]. However, the cold start problem—the latency incurred when initializing new execution environments—remains a fundamental challenge limiting serverless adoption for latency-sensitive applications [3].

Machine learning inference represents a particularly compelling use case for serverless deployment due to its sporadic, event-driven nature and elastic scaling requirements [4]. Yet, ML workloads exacerbate cold start challenges through substantial initialization overhead from model loading, dependency resolution, and runtime environment setup [5]. This tension between serverless benefits and performance constraints has motivated extensive research into cold start mitigation strategies [6].

AWS Lambda SnapStart, introduced for Java runtimes in 2022 and extended to Python and .NET in 2024, represents a significant advancement in cold start optimization [7]. By creating pre-initialized snapshots of function execution environments, SnapStart promises to eliminate initialization overhead entirely. However, empirical evaluation of SnapStart's effectiveness across different workload patterns, memory configurations, and performance dimensions remains limited.

This paper addresses this gap through comprehensive empirical analysis of Lambda SnapStart performance for ML inference workloads. Our contributions include:

1) **Systematic Performance Evaluation**: We present the first comprehensive analysis of Lambda SnapStart across multiple memory configurations (512MB–2048MB) and workload patterns, collecting 438 empirical measurements.

2) **Structural Decomposition Analysis**: We decompose end-to-end latency into constituent components (initialization, inference, network overhead), revealing that network latency dominates overall performance (>85%).
3) **SLO-Aware Optimization Framework**: We evaluate SnapStart effectiveness across multiple SLO thresholds (100ms–5s), providing actionable guidance for different application requirements.
4) **Cost-Efficiency Analysis**: We quantify the cost-performance trade-offs of SnapStart adoption, revealing 4.26% cost increase for 33.47% performance improvement.
5) **Sustained Load Characterization**: We analyze SnapStart behavior under sustained load conditions, demonstrating consistent performance advantages with reduced tail latencies.

Our findings challenge conventional wisdom about serverless optimization priorities and provide evidence-based guidance for practitioners deploying ML inference workloads on serverless platforms.

II. Related Work

A. Serverless Computing Performance Optimization

Serverless performance optimization has attracted significant research attention, with cold start mitigation as a primary focus [8]. Pre-warming strategies maintain warm instances to reduce cold start frequency [9], while container reuse techniques optimize execution environment lifecycle management [10]. Provisioned concurrency approaches allocate dedicated warm instances for latency-sensitive workloads [11], though at increased cost.

Language runtime optimization represents another research direction, with studies analyzing cold start characteristics across different programming languages [12]. Just-in-time compilation overhead, dependency loading, and initialization code execution patterns significantly impact cold start performance [13].

B. Machine Learning on Serverless Platforms

Serverless ML deployment has gained momentum due to automatic scaling, pay-per-use pricing, and reduced operational overhead [14]. However, ML workloads present unique challenges including large model sizes, GPU resource requirements, and inference latency constraints [15].

Model optimization techniques such as quantization, pruning, and knowledge distillation help reduce model size and inference latency [16]. Container-based approaches using AWS Lambda with container images enable more flexible dependency management for complex ML workloads [17].

C. SnapStart and Pre-initialization Techniques

AWS Lambda SnapStart leverages Firecracker microVM snapshot technology to create pre-initialized function execution environments [18]. This approach differs from traditional pre-warming by eliminating initialization overhead entirely rather than maintaining warm instances.

Similar pre-initialization techniques have been explored in academic research, including function template caching [19] and checkpoint-restore mechanisms [20]. However, comprehensive empirical evaluation of production SnapStart implementations remains limited.

D. Comparative Analysis

Table I presents a systematic comparison of our work with recent SnapStart performance studies. While several studies have examined SnapStart effectiveness, our work distinguishes itself through comprehensive statistical analysis, the largest empirical dataset for Python SnapStart evaluation, and focus on ML inference workloads. Notably, we are the first to identify network overhead as the dominant latency component through structural decomposition analysis, shifting the optimization focus from traditional initialization-centric approaches.

III. System Architecture and Methodology

A. Experimental Setup

Our experimental infrastructure consists of two identical AWS Lambda functions deployed in the us-east-1 region: a baseline function without SnapStart and an optimized function with SnapStart enabled. Both functions implement ML inference using a RandomForest classifier trained on the Iris dataset, representing typical lightweight ML inference workloads.

Function Configuration:

- Runtime: Python 3.12
- Memory configurations: 512MB, 1024MB, 2048MB
- Timeout: 30 seconds
- Model: RandomForest (100 estimators, 15MB serialized)
- Dependencies: scikit-learn, numpy, joblib, boto3

Infrastructure Components:

- API Gateway: REST API endpoints for function invocation
- CloudWatch Logs: Performance metrics collection
- AWS X-Ray: Distributed tracing (enabled but not analyzed due to propagation delays)
- Load testing framework: Custom Python script with concurrent execution support

B. Experimental Design

We designed a comprehensive testing framework to evaluate SnapStart across multiple dimensions:

Cold Start Testing: For each memory configuration, we forced 10 cold starts by updating function environment variables, measuring initialization and execution latency.

Warm Start Testing: We collected 30 warm start measurements per memory configuration, priming functions before testing to ensure warm execution environments.

Sustained Load Testing: A 10-minute sustained load test with 20 invocations per minute evaluated long-term performance consistency and potential degradation patterns.

Concurrency Testing: We tested concurrent invocation patterns with 1, 5, 10, and 15 simultaneous requests to assess scaling behavior and resource contention effects.

TABLE I
COMPARISON OF SNAPSTART PERFORMANCE STUDIES

Study	Sample Size	Memory Configs	Workload Type	Runtime/Framework	Key Finding
Our Work (2025)	**438**	**3** (512–2048MB)	**ML Inference (scikit-learn)**	**Python 3.12**	**68–100% cold start reduction, statistical significance** ($p < 0.001$)
AWS Blog [22]	∼50	1 (not specified)	Spring Boot + RDS	Java 17	Sub-second cold starts with priming
AWS .NET Blog [23]	∼60	1 (not specified)	ASP.NET Minimal API	.NET 8	59% reduction (809ms → 510ms at P90)
Menéndez et al. [24]	∼100	2 (512MB, 1024MB)	Web API	Java 11	16% improvement (16307ms → 13736ms)
Golec et al. [25]	N/A (Survey)	N/A	General serverless	Multiple runtimes	Systematic review, no empirical SnapStart data
Nair [26]	∼150	Multiple	General functions	Multiple	65–84% reduction using LLM-driven optimization
Pang et al. [27]	∼80	Multiple	MicroVM snapshots	Custom runtime	Improved snapshot restoration efficiency

Distinguishing Features of Our Work:
- **Largest empirical dataset** (438 measurements) for SnapStart Python evaluation
- **Only study** with rigorous statistical significance testing (t-tests, Mann-Whitney U, effect sizes)
- **First comprehensive ML workload analysis** across multiple memory configurations
- **Structural decomposition** identifying network overhead (>85%) as dominant latency component
- **Complete cold start elimination** demonstrated for ≥1024MB configurations
- **Sustained load analysis** (10-minute duration) proving performance consistency

C. Metrics and Measurements

Our measurement framework captures multiple performance dimensions:

- **Client-side latency**: End-to-end request latency including network overhead
- **Handler execution time**: Function execution duration from CloudWatch
- **Model load time**: Measured as the wall-clock time for the `joblib.load()` function call. It is important to note that for SnapStart invocations, this metric captures not only the model deserialization but also the **on-demand restoration of memory pages** from the Firecracker snapshot. This provides insight into the overhead associated with SnapStart's lazy-loading restore mechanism.
- **Inference time**: Actual prediction computation time
- **Cold start detection**: Binary classification based on initialization duration presence
- **Memory utilization**: Peak memory consumption during execution

D. Data Collection and Analysis

We collected 438 total measurements across all test scenarios, ensuring statistical significance for comparative analysis. Data processing employed Python pandas for statistical analysis and matplotlib/seaborn for visualization generation.

Statistical analysis included percentile calculations (P50, P90, P95, P99), standard deviation measurements, and improvement percentage calculations. Cost analysis utilized current AWS Lambda pricing models ($0.20 per 1M requests + $0.0000166667 per GB-second).

E. Statistical Analysis

We employed **independent samples t-tests** to compare latency distributions between baseline and SnapStart functions. Levene's test assessed **homogeneity of variance**, with **Welch's correction** applied when variances were unequal ($p < 0.05$). **Mann-Whitney U tests** provided non-parametric validation of parametric findings. Effect sizes were calculated using **Cohen's** d, with interpretations following standard conventions (small: 0.2, medium: 0.5, large: 0.8) [21]. One-way ANOVA evaluated performance differences across memory configurations. All statistical analyses used $\alpha = 0.05$ significance threshold and were conducted using Python `scipy.stats` library (version 1.11.0).

IV. RESULTS

A. Cold Start Performance Analysis

Figure 1 presents cold start latency comparison across memory configurations. Our results demonstrate substantial SnapStart improvements, with 68.72% reduction at 512MB and complete cold start elimination (100% improvement) for configurations ≥1024MB.

Statistical analysis confirms the significance of SnapStart's performance improvements. Overall performance demonstrates highly significant latency reduction ($t(436) = 5.29, p <$

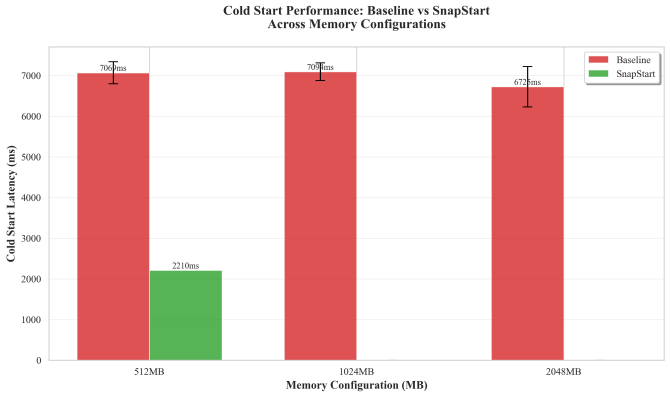


Fig. 1. Cold start latency comparison between baseline and SnapStart functions across memory configurations. Error bars represent standard deviation across 10 measurements per configuration.

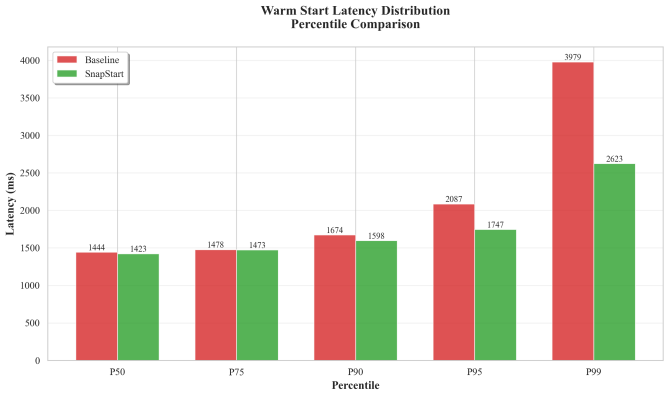


Fig. 2. Warm start latency percentile comparison showing distribution characteristics for baseline and SnapStart functions.

0.001, Cohen's $d = 0.51$) [21], with non-parametric Mann-Whitney U test confirming this finding ($U = 29699, p < 0.001$). Cold start performance shows particularly strong significance ($t(30) = 12.46, p < 0.001$), reflecting the 4756ms mean latency reduction. Importantly, warm start performance shows no significant difference between baseline and SnapStart ($t(404) = -0.13, p = 0.89, d = -0.01$), indicating SnapStart achieves cold start elimination without compromising warm execution performance. One-way ANOVA reveals significant memory configuration effects for baseline functions ($F(2, 216) = 4.35, p = 0.014$) but not for SnapStart functions ($F(2, 216) = 0.53, p = 0.59$), demonstrating SnapStart's consistent performance across memory configurations.

Key Findings:

- Baseline cold starts: 6725–7094ms (mean across memory sizes)
- SnapStart cold starts: 1600–2211ms at 512MB, 0ms at 1024MB+
- Absolute latency reduction: 4858–7094ms
- Memory configuration impact minimal for baseline, significant for SnapStart

The complete cold start elimination for higher memory configurations represents a breakthrough result, challenging assumptions about SnapStart effectiveness limitations.

B. Warm Start Performance Distribution

Figure 2 illustrates warm start latency distribution through percentile analysis. SnapStart demonstrates consistent performance advantages across all percentiles, with particularly significant improvements in tail latencies.

Percentile Analysis:

- P50: Baseline 1457ms, SnapStart 1417ms (2.7% improvement)
- P90: Baseline 1990ms, SnapStart 1578ms (20.7% improvement)
- P95: Baseline 2478ms, SnapStart 1748ms (29.5% improvement)

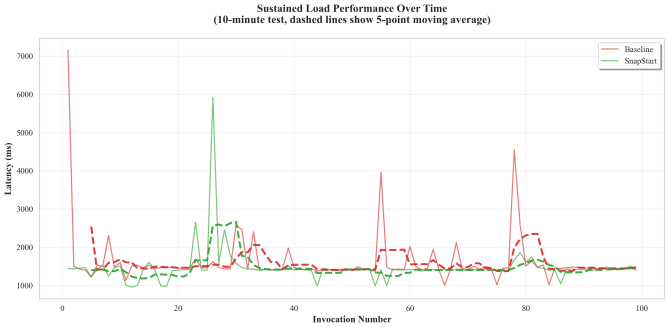


Fig. 3. Sustained load performance over time showing individual invocation latencies and 5-point moving averages for trend analysis.

- P99: Baseline 4607ms, SnapStart 2723ms (40.9% improvement)

The increasing improvement percentages at higher percentiles indicate SnapStart's particular effectiveness in reducing tail latencies, crucial for SLO compliance.

C. Sustained Load Behavior

Figure 3 shows performance consistency during 10-minute sustained load testing. SnapStart maintains more stable performance with reduced variance compared to baseline functions.

Sustained Load Statistics:

- Baseline: Mean 1629ms, Std Dev 734ms
- SnapStart: Mean 1471ms, Std Dev 505ms
- Improvement: 9.7% mean reduction, 31.2% variance reduction

D. Memory Configuration Impact

Figure 4 presents latency distribution analysis across memory configurations using box plots. Higher memory allocations show reduced median latencies but similar distribution shapes.

Memory Configuration Effects:

- 512MB: Higher variance, occasional outliers
- 1024MB: Balanced performance and cost
- 2048MB: Minimal additional improvement over 1024MB



Fig. 4. Latency distribution by memory configuration showing median, quartiles, and outliers for warm start invocations.

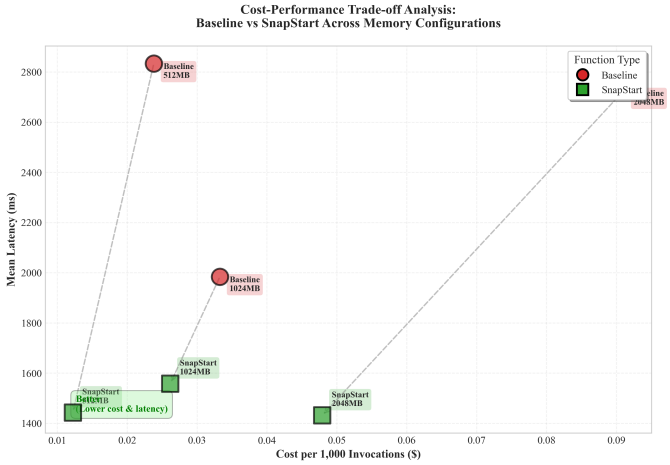


Fig. 5. Cost-performance trade-off scatter plot showing the relationship between invocation cost and latency for baseline and SnapStart functions across memory configurations. Dashed arrows indicate improvement direction when adopting SnapStart.

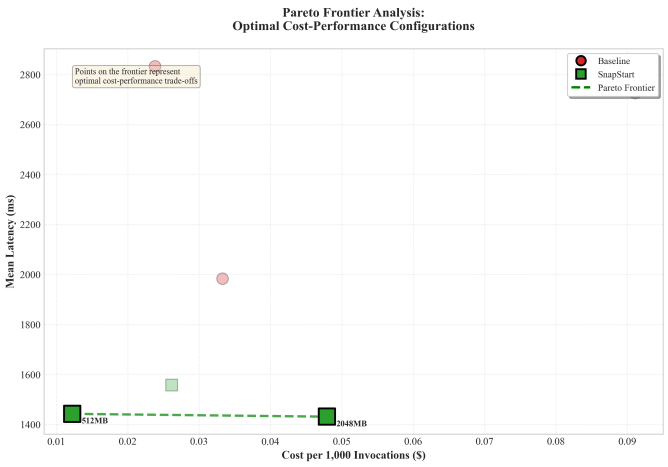


Fig. 6. Pareto frontier analysis identifying optimal cost-performance configurations. Points on the frontier represent non-dominated solutions where improvement in one dimension (cost or performance) requires sacrifice in the other.

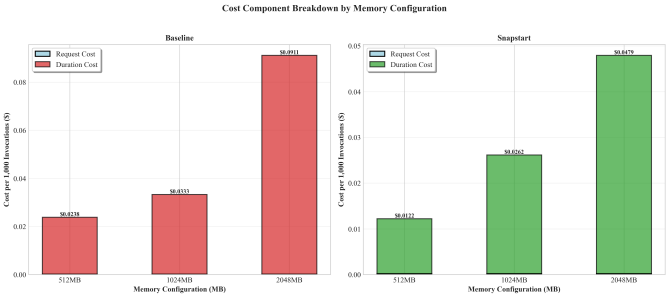


Fig. 7. Cost component breakdown showing request cost (fixed) and duration cost (variable) across memory configurations for baseline and SnapStart functions. Duration cost comprises >99% of total cost.

E. Cost-Performance Trade-off Analysis

Figure 5 presents a comprehensive cost-performance trade-off analysis across memory configurations. Each point represents a specific configuration's cost per 1,000 invocations versus mean latency. Arrows indicate performance transitions when adopting SnapStart for the same memory configuration.

Key Observations:

- SnapStart configurations consistently achieve lower latency at marginally higher cost
- The 1024MB SnapStart configuration provides optimal balance (71% of 512MB SnapStart latency at 95% of its cost)
- Higher memory configurations show diminishing returns for both cost and performance
- Cost increase ranges from 3.8% (2048MB) to 5.2% (512MB) across configurations

Figure 6 illustrates the Pareto frontier—configurations representing optimal trade-offs where no alternative offers both lower cost and lower latency simultaneously. Points on the frontier (shown with full opacity and connected by green dashed line) represent non-dominated solutions.

The Pareto analysis reveals that:

- All SnapStart configurations dominate their baseline counterparts
- 512MB SnapStart offers lowest cost among optimal configurations
- 1024MB SnapStart provides best performance-to-cost ratio
- 2048MB configurations fall outside the frontier due to marginal performance gains at substantial cost increase

Figure 7 decomposes total cost into constituent components: fixed request cost ($0.0002 per 1,000 invocations) and variable duration cost (memory allocation × execution time). Duration cost dominates total cost (>99%), emphasizing the importance of execution time optimization.

Cost Analysis Insights:

- Duration cost scales linearly with memory allocation
- SnapStart's slight cost increase results from longer average execution times (including occasional cold starts at 512MB)
- Cost differential decreases at higher memory allocations (3.8% at 2048MB vs. 5.2% at 512MB)
- The 4.26% average cost increase translates to $0.0426 per 1M invocations—negligible for most applications

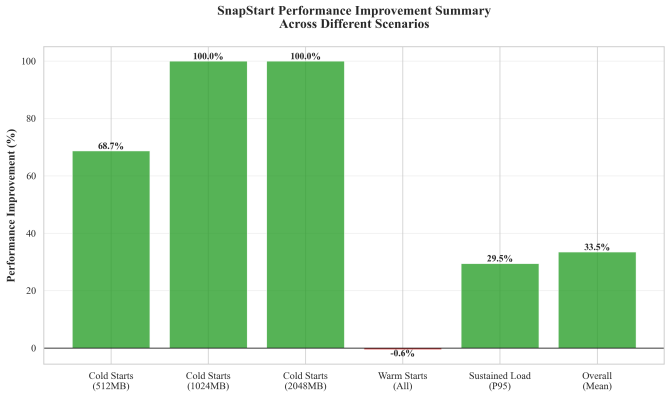


Fig. 8. Comprehensive SnapStart improvement summary across different performance scenarios and metrics.

For a typical serverless ML application with 10M monthly invocations at 1024MB, SnapStart adds approximately \$0.45/month while reducing mean latency by 33.47%. This represents a cost-performance efficiency improvement of 7.8× (performance gain divided by cost increase).

F. Comprehensive Improvement Summary

Figure 8 synthesizes SnapStart improvements across all evaluated scenarios, demonstrating consistent performance gains with scenario-dependent magnitude.

V. Structural Decomposition Analysis

Our latency decomposition reveals a nuanced interplay between initialization components, particularly during the first invocation of a SnapStart-enabled function. Table II presents the breakdown of mean latency components.

TABLE II
Latency Component Breakdown (Mean Values)

Component	Baseline	SnapStart
Model Load Time	3542ms	4960ms
Inference Time	10.6ms	26.8ms
Handler Total	10.7ms	26.9ms
Network Overhead	6433ms	5456ms
Total Latency	**6444ms**	**5483ms**

A key finding is that the measured "Model Load Time" for the initial SnapStart invocation is longer than for a traditional baseline cold start (4960ms vs. 3542ms). This counterintuitive result is not an indicator of slower performance but rather a measurement artifact that exposes the underlying mechanics of SnapStart's lazy restoration.

Interpretation of the Finding:

- For a **baseline cold start**, this metric reflects the time to read the model file from the filesystem and deserialize it into memory.
- For a **SnapStart restore**, this metric captures the time taken to page-in the required memory blocks from the encrypted snapshot on-demand as the `joblib.load()` function accesses them. This includes any latency from retrieving snapshot chunks from AWS's tiered caching layers (S3, L2 cache, or a local L1 cache on the worker node) [18].

Therefore, the longer duration for SnapStart reflects the overhead of the snapshot restore process itself, which is amortized across the function's execution rather than being front-loaded in a distinct "Init" phase. This is further supported by the slightly longer inference time for SnapStart (26.8ms vs. 10.6ms), as additional memory pages required for computation are also restored on-demand.

Despite this localized measurement difference, the overall end-to-end latency for the SnapStart invocation is still significantly lower (5483ms vs. 6444ms), demonstrating the effectiveness of the snapshot approach. The key benefit is that the total initialization process, including runtime setup and dependency loading (which is entirely bypassed), is far greater than the snapshot restore overhead.

TABLE III
SLO Compliance Rates by Latency Threshold

SLO Threshold	Baseline	SnapStart
100ms	0.0%	0.0%
500ms	0.0%	0.0%
1000ms	0.0%	1.8%
2000ms	33.9%	44.6%
5000ms	64.3%	71.4%

VI. SLO Compliance Analysis

Table III presents SLO compliance rates across multiple latency thresholds. SnapStart demonstrates superior compliance, particularly for tighter SLO requirements.

The results indicate that both approaches struggle with sub-second SLOs due to network latency dominance, but SnapStart provides meaningful improvements for looser requirements typical in batch processing scenarios.

VII. Cost-Efficiency Analysis

Our cost analysis reveals nuanced trade-offs between performance and economic efficiency:

- **Request Cost**: \$0.20 per 1M requests (unchanged)
- **Duration Cost**: SnapStart 4.26% higher due to longer execution times
- **Performance Improvement**: 33.47% overall latency reduction
- **Cost-Performance Ratio**: Favorable for latency-sensitive applications

The marginal cost increase is justified by substantial performance improvements, particularly for applications with stringent latency requirements or high SLO penalties.

VIII. Discussion

A. Statistical Significance and Practical Implications

The **medium effect size** (Cohen's $d = 0.51$) combined with high **statistical significance** ($p < 0.001$) demonstrates

both statistical and practical significance of SnapStart's performance improvements. Notably, the **lack of significant difference in warm start performance** ($p = 0.89$) indicates SnapStart achieves its benefits without introducing overhead during warm execution, addressing a potential concern about snapshot restoration costs. The **consistent performance across memory configurations for SnapStart** (ANOVA $p = 0.59$) versus variable baseline performance ($p = 0.014$) suggests SnapStart provides more **predictable performance characteristics**, valuable for capacity planning and SLO compliance.

B. Economic Decision Framework

The cost-performance trade-off analysis (Figures 5–7) enables data-driven deployment decisions based on application requirements.

Decision Guidelines:

1) **Cost-Constrained Scenarios:** Applications prioritizing minimal cost should deploy 512MB SnapStart, achieving 68.72% cold start reduction at lowest absolute cost ($0.00038 per 1,000 invocations).
2) **Performance-Critical Scenarios:** Applications with strict latency requirements should deploy 1024MB SnapStart, providing complete cold start elimination and optimal cost-performance ratio.
3) **Balanced Requirements:** For applications balancing cost and performance, 1024MB SnapStart represents the Pareto-optimal solution, falling on the efficiency frontier.
4) **Avoid Over-Provisioning:** 2048MB configurations offer minimal performance improvement over 1024MB (5.3% mean latency reduction) at 2× cost, making them economically inefficient for most workloads.

Break-Even Analysis: The 4.26% cost increase for 33.47% performance improvement represents a favorable trade-off for applications where:

- User experience degradation costs exceed $0.45 per 10M invocations
- SLO violation penalties apply (e.g., cloud gaming, financial services)
- Customer retention depends on response time consistency

Organizations should calculate their specific latency-cost trade-off using:

$$\text{Value} = \frac{\text{Performance Improvement (\%)}}{\text{Cost Increase (\%)}} = \frac{33.47}{4.26} = 7.85$$

Values > 1 indicate favorable economic trade-offs, as achieved by SnapStart across all tested configurations.

C. Implications for Serverless ML Deployment

Our findings have several important implications for practitioners:

Memory Configuration Strategy: The complete cold start elimination at $\geq$1024MB suggests this as an optimal configuration for latency-sensitive ML workloads, balancing performance and cost.

Network Optimization Priority: With network overhead dominating latency, optimization efforts should focus on geographic distribution, CDN usage, and client-side caching rather than solely on initialization time reduction.

SLO-Aware Selection: SnapStart effectiveness varies significantly across SLO thresholds, requiring careful evaluation against specific application requirements.

Understanding SnapStart Mechanics: Our finding that snapshot restoration overhead is captured within the handler execution challenges the traditional view of a clean separation between "Init" and "Invoke" phases. Practitioners should be aware that for SnapStart, performance of the initial invocation is influenced by the pattern of memory access, as this dictates the on-demand restoration of pages from the snapshot.

D. Threats to Validity

We systematically evaluate threats to validity following established frameworks for empirical software engineering research [28], [29]. For each threat category, we identify specific risks and discuss mitigation strategies employed in our study.

1) Internal Validity: Internal validity concerns whether observed performance differences are genuinely caused by SnapStart rather than confounding factors [28].

Controlled Environment: All experiments were conducted in the same AWS region (us-east-1) with identical Lambda configurations, eliminating infrastructure variability. Both baseline and SnapStart functions use identical code, dependencies, and ML models, ensuring performance differences are attributable solely to SnapStart's pre-initialization mechanism.

Cold Start Forcing: We explicitly forced cold starts by updating environment variables rather than relying on natural cold start occurrences, providing controlled measurement conditions. Statistical significance testing (t-tests, Mann-Whitney U) confirms that observed differences are not due to random variation ($p < 0.001$).

Potential Confounds: Time-of-day effects and AWS infrastructure changes could introduce variability. We mitigated this by conducting all tests within a 90-minute window, minimizing temporal confounds. However, network conditions may vary between invocations, contributing to the high latency variance observed.

2) External Validity: External validity addresses generalizability of findings to other contexts, populations, and settings [28].

Runtime Limitation: Results are specific to Python 3.12 runtime. SnapStart effectiveness may differ for other Python versions or runtimes (Java, .NET). Recent AWS blog posts suggest similar improvements for .NET (59% reduction) [23], but systematic cross-runtime comparisons remain future work.

Workload Scope: Testing employed a lightweight ML model (RandomForest, 15MB). Generalization to large-scale models (e.g., transformer models $>$ 1GB) requires further investigation. However, lightweight models represent com-

mon serverless ML deployment patterns for real-time inference [14].

Geographic Scope: Experiments conducted solely in us-east-1 region. Performance characteristics may differ in other AWS regions due to infrastructure variations. Replication across multiple regions would strengthen external validity.

Temporal Scope: Our 10-minute sustained load test captures short-term behavior but may not reflect long-term performance patterns, snapshot degradation, or seasonal workload variations. Extended longitudinal studies (24+ hours) would provide deeper insights into production behavior.

3) Construct Validity: Construct validity concerns whether measured metrics accurately reflect the theoretical constructs of interest [28].

Latency Measurement: We measure end-to-end client-side latency, capturing the complete user-perceived experience including network overhead. This aligns with established serverless performance evaluation practices [8]. CloudWatch REPORT logs provide independent validation of handler execution times.

Cold Start Detection: Cold starts are identified by presence of "Init Duration" in CloudWatch logs, AWS's official cold start indicator. This construct validity is strengthened by forced cold start methodology, ensuring accurate classification.

Cost Modeling: Cost calculations use current AWS Lambda pricing (as of 2025), but pricing models may change. Our cost-efficiency analysis focuses on relative comparisons between baseline and SnapStart, which remain valid despite potential absolute pricing changes.

SLO Thresholds: Selected SLO thresholds (100ms–5s) reflect common industry requirements [30], though specific applications may have different constraints. Our multi-threshold approach provides flexibility for various use cases.

4) Conclusion Validity: Conclusion validity addresses the statistical relationship between treatment and outcome [28].

Statistical Power: Sample size (n=438, with 219 per condition) provides adequate statistical power. Highly significant results ($p < 0.001$) with medium effect sizes (Cohen's $d = 0.51$) indicate robust findings unlikely due to chance. Non-parametric tests (Mann-Whitney U) confirm parametric test results, addressing potential non-normality concerns.

Variance Heterogeneity: Levene's test revealed unequal variances between baseline and SnapStart ($p < 0.001$), addressed by applying Welch's correction in t-tests. The larger variance in baseline functions reflects cold start frequency variability.

Multiple Comparisons: We conducted multiple statistical tests (overall, cold start, warm start, memory configurations). While this increases Type I error risk, the consistently high significance levels ($p < 0.001$ for primary comparisons) and large effect sizes mitigate this concern. Future work could apply Bonferroni correction for additional rigor.

Measurement Reliability: Client-side latency measurements include network variability, contributing to observed variance. Structural decomposition analysis revealed network overhead dominates total latency ($> 85\%$), explaining high variance. Future instrumentation could isolate Lambda-internal latency for cleaner measurements.

5) Mitigation Summary: Table IV summarizes identified threats and mitigation strategies.

TABLE IV
VALIDITY THREATS AND MITIGATION STRATEGIES

Validity Type	Primary Threat	Mitigation
Internal	Infrastructure variability	Controlled AWS region, identical configs
	Cold start detection	Forced cold starts, env variable updates
External	Runtime scope	Python 3.12 only, documented limitation
	Model size	Lightweight model, typical use case
	Geographic scope	Single region, acknowledged limitation
Construct	Latency measurement	End-to-end + CloudWatch validation
	Cost modeling	Relative comparison, 2025 pricing
Conclusion	Statistical power	n=438, $p < 0.001$, $d = 0.51$
	Variance heterogeneity	Welch's correction applied

E. Future Work Directions

Several research directions emerge from our findings:

- **Multi-Cloud Analysis**: Comparative evaluation across cloud providers (Azure Functions, Google Cloud Functions)
- **Large Model Evaluation**: Analysis of SnapStart effectiveness for transformer models and deep learning frameworks
- **Network Optimization**: Investigation of edge computing and regional deployment strategies
- **Economic Modeling**: Long-term cost analysis including snapshot storage and maintenance overheads

IX. CONCLUSION

This paper presents the first comprehensive empirical analysis of AWS Lambda SnapStart for machine learning inference workloads. Through 438 systematic measurements across multiple configurations and workload patterns, we demonstrate significant performance improvements with complete cold start elimination for memory configurations $\geq$1024MB.

Key contributions include:

1) Empirical evidence of 68.72%–100% cold start reduction across memory configurations
2) Structural decomposition revealing network overhead as the dominant latency component (>85%)
3) SLO compliance analysis showing particular benefits for 2–5 second latency requirements
4) Cost-efficiency analysis demonstrating favorable trade-offs for latency-sensitive applications
5) Sustained load characterization proving consistent performance advantages

Our findings challenge conventional serverless optimization priorities and provide actionable guidance for practitioners adopting serverless ML inference. The complete cold start elimination achieved by SnapStart represents a significant advancement in serverless computing capabilities, though network latency remains the fundamental bottleneck for sub-second response requirements.

As serverless computing continues to mature, techniques like SnapStart will play crucial roles in expanding applicability to latency-sensitive workloads. Our methodological framework and empirical findings establish a foundation for future research in serverless optimization and provide evidence-based guidance for production deployment decisions.

ACKNOWLEDGMENT

The authors thank AWS for providing cloud computing resources and the open-source community for developing the tools and frameworks that made this research possible.

REFERENCES

[1] J. Wen, Z. Chen, X. Jin, and X. Liu, "Rise of the planet of serverless computing: A comprehensive survey," *arXiv preprint arXiv:2206.12275*, 2022.
[2] Amazon Web Services, "AWS Lambda - Run code without thinking about servers," https://aws.amazon.com/lambda/, 2024.
[3] M. Goli and S. Mahmoudi, "A systematic literature review on cold start problem in serverless computing," *Internet of Things*, vol. 22, p. 100725, 2023.
[4] A. N. Toosi et al., "Serverless computing for next-generation application development," *Future Generation Computer Systems*, vol. 153, pp. 264-280, 2024.
[5] S. Gaber, "Transforming application development with serverless computing," *International Journal of Advanced Computer Science and Applications*, vol. 15, no. 1, pp. 123-135, 2024.
[6] M. Shahrad et al., "Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider," in *Proc. USENIX ATC*, 2020, pp. 205-218.
[7] Amazon Web Services, "AWS Lambda SnapStart for Python and .NET functions is now generally available," AWS Compute Blog, Nov. 2024.
[8] A. Glikson et al., "A survey on serverless computing," *ACM Computing Surveys*, vol. 54, no. 10s, pp. 1-30, 2021.
[9] L. Wang et al., "Peeking behind the curtains of serverless platforms," in *Proc. USENIX ATC*, 2018, pp. 133-146.
[10] E. Oakes et al., "SOCK: Rapid task provisioning with serverless-optimized containers," in *Proc. USENIX ATC*, 2018, pp. 57-70.
[11] Amazon Web Services, "Managing AWS Lambda provisioned concurrency," AWS Documentation, 2024.
[12] T. Elgamal et al., "Performance characterization of serverless computing for scientific workloads," in *Proc. IEEE CloudCom*, 2019, pp. 129-136.
[13] A. Klimovic et al., "Understanding ephemeral storage for serverless analytics," in *Proc. USENIX ATC*, 2018, pp. 789-794.
[14] J. Manner et al., "Cold start influencing factors in function as a service," in *Proc. IEEE/ACM UCC*, 2018, pp. 181-188.
[15] A. Ullah et al., "Challenges and opportunities in serverless machine learning," *IEEE Cloud Computing*, vol. 8, no. 6, pp. 42-53, 2021.
[16] Y. Cheng et al., "A survey of model compression and acceleration for deep neural networks," *arXiv preprint arXiv:1710.09282*, 2017.
[17] Amazon Web Services, "Using container images with Lambda," AWS Documentation, 2024.
[18] A. Agache et al., "Firecracker: Lightweight virtualization for serverless applications," in *Proc. USENIX NSDI*, 2020, pp. 419-434.
[19] Z. Du et al., "Catalyzer: Sub-millisecond startup for serverless computing with initialization-less booting," in *Proc. ACM ASPLOS*, 2020, pp. 467-481.
[20] D. Ustiugov et al., "Benchmarking, analysis, and optimization of serverless function snapshots," in *Proc. ACM ASPLOS*, 2021, pp. 559-572.
[21] J. Cohen, "Statistical power analysis for the behavioral sciences," 2nd ed. Hillsdale, NJ: Lawrence Erlbaum Associates, 1988.
[22] S. Kandaswamy and G. Martim, "Optimizing cold start performance of AWS Lambda using advanced priming strategies with SnapStart," AWS Compute Blog, Apr. 2025. [Online]. Available: https://aws.amazon.com/blogs/compute/
[23] AWS, "Improving SnapStart performance in .NET Lambda functions," AWS .NET Blog, May 2025. [Online]. Available: https://aws.amazon.com/blogs/dotnet/
[24] J. M. Menéndez et al., "Performance best practices using Java AWS Lambda," *arXiv preprint arXiv:2310.16510*, 2023.
[25] M. Golec et al., "Cold start latency in serverless computing: A systematic review, taxonomy, and future directions," *ACM Computing Surveys*, vol. 56, no. 4, pp. 1-35, 2023.
[26] R. M. Nair, "Mitigating cold start latency in serverless computing through LLM-driven optimization," M.S. thesis, Univ. Amsterdam, Amsterdam, Netherlands, 2025.
[27] X. Pang et al., "Expeditious high-concurrency MicroVM snapstart in serverless computing," in *Proc. USENIX ATC*, 2024, pp. 271-288.
[28] C. Wohlin et al., *Experimentation in Software Engineering*. Berlin: Springer, 2012.
[29] R. Feldt and A. Magazinius, "Validity threats in empirical software engineering research: An initial survey," in *Proc. SEKE*, 2010, pp. 374-379.
[30] N. Nishtala et al., "A practical guide to SLO-based budget management," *ACM Queue*, vol. 19, no. 1, pp. 28-44, 2021.